Tools Options Help

Sans 12

Asymptotic Notation

ayer: Layer 1

Tools Options Help
Sans 12

Representing "Rough" by Asymptotic Notation

- goal: rough rather than exact steps
- why rough? constant not matter much

—when input size large

compare two complexity functions $f(n)$ and $g(n)$

growth of functions matters

—when n large, n^3 eventually bigger than $1126n$

rough $\Leftrightarrow$ asymptotic behavior

Layer: Layer 1

Tools Options Help
Sans 12

Asymptotic Notations: Rough Upper Bound

big-O: rough upper bound

- $f(n)$ grows slower than or similar to $g(n)$: $f(n) = O(g(n))$
  - n grows slower than n^2: $n = O(n^2)$
  - $3n$ grows similar to n: $3n = O(n)$
- asymptotic intuition (rigorous math later):

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} \leq c$$

big-O: arguably the most used "language" for complexity

Layer: Layer 1

Representing "Rough" by Asymptotic Notation

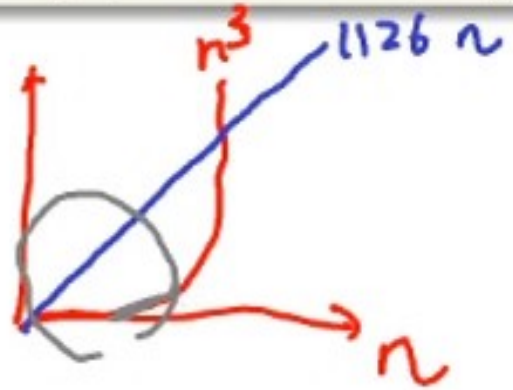
- goal: rough rather than exact steps
- why rough? constant not matter much

—when input size large

compare two complexity functions $f(n)$ and $g(n)$

growth of functions matters

—when n large, n^3 eventually bigger than $1126n$



rough $\Leftrightarrow$ asymptotic behavior

Asymptotic Notations: Rough Upper Bound

big-O: rough upper bound

- $f(n)$ grows slower than or similar to $g(n)$: $f(n) = O(g(n))$
  - n grows slower than n^2: $n = O(n^2)$
  - $3n$ grows similar to n: $3n = O(n)$
- asymptotic intuition (rigorous math later):

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} \leq c$$

big-O: arguably the most used "language" for complexity

Tools Options Help
Sans 12

More Intuitions on Big-O

$$f(n) = O(g(n)) \Leftarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} \le c \quad \text{(not rigorously, yet)}$$

- "$= O(\cdot)$" more like "$\in$"
  - $n = O(n)$
  - $n = O(10n)$
  - $n = O(0.3n)$
  - $n = O(n^5)$
- "$= O(\cdot)$" also like "$\le$"
  - $n = O(n^2)$
  - $n^2 = O(n^{2.5})$
  - $n = O(n^{2.5})$
- $1126n = O(n)$: coefficient not matter
- $n + \sqrt{n} + \log n = O(n)$: lower-order term not matter

intuitions (properties) to be proved later

Layer: Layer 1

Tools Options Help
Sans 12

More Intuitions on Big-O

$$f(n) = O(g(n)) \Leftarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} \le c \quad \text{(not rigorously, yet)}$$

1126

$\sqrt{n}$

n

$n^2 = O(n^2)$

$0.1n^2$

$3n^2$

- "$= O(\cdot)$" more like "$\in$"
  - $n = O(n)$
  - $n = O(10n)$
  - $n = O(0.3n)$
  - $n = O(n^5)$
- "$= O(\cdot)$" also like "$\le$"
  - $n = O(n^2)$
  - $n^2 = O(n^{2.5})$
  - $n = O(n^{2.5})$
- $1126n = O(n)$: coefficient not matter
- $n + \sqrt{n} + \log n = O(n)$: lower-order term not matter

"exact" intuitions (properties) to be proved later

Layer: Layer 1

Formal Definition of Big-O

$\lim \frac{f}{g} \leq C$

$f(n) > 0$

$g(n) > 0$

Consider positive functions $f(n)$ and $g(n)$,

$f(n) = O(g(n))$, iff exist c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

- covers the lim intuition if limit exists
- covers other situations without "limit"
  e.g. $|\sin(n)| = O(1)$

next: prove that lim intuition $\Rightarrow$ formal definition

Tools Options Help
Sans 12

lim Intuition ⇒ Formal Definition

For positive functions f and g, if $\lim_{n\to\infty} \frac{f(n)}{g(n)} \le c$, then $f(n) = O(g(n))$.

- with definition of limit, there exists ϵ, n_0 such that for all $n \ge n_0$, $|\frac{f(n)}{g(n)} - c| < \epsilon$.
- That is, for all $n \ge n_0$, $\frac{f(n)}{g(n)} < c + \epsilon$.
- Let $c' = c + \epsilon$, $n_0' = n_0$, big-O definition satisfied with (c', n_0'). QED.

important to not just have intuition (building),
but know definition (building block)

Layer: Layer 1

lim Intuition ⇒ Formal Definition

$f \le c' g$
(c', n_0')

For positive functions f and g, if $\lim_{n\to\infty} \frac{f(n)}{g(n)} \le c$, then $f(n) = O(g(n))$.

- with definition of limit, there exists ϵ, n_0 such that for all $n \ge n_0$, $|\frac{f(n)}{g(n)} - c| < \epsilon$.
- That is, for all $n \ge n_0$, $\frac{f(n)}{g(n)} < c + \epsilon$.
- Let $c' = c + \epsilon$, $n_0' = n_0$, big-O definition satisfied with (c', n_0'). QED.

important to not just have intuition (building),
but know definition (building block)

Asymptotic Notations: Definitions

- $f(n)$ grows slower than or similar to $g(n)$: ("$\leq$")

  $f(n) = O(g(n))$, iff exist c, n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

- $f(n)$ grows faster than or similar to $g(n)$: ("$\geq$")

  $f(n) = \Omega(g(n))$, iff exist c, n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$

- $f(n)$ grows similar to $g(n)$: ("$\approx$")

  $f(n) = \Theta(g(n))$, iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

let's see how to use them

The Seven Functions as g

$g(n) = ?$

- 1: constant
- $\log n$: logarithmic (does base matter?)
- n: linear
- $n \log n$
- n^2: square
- n^3: cubic
- 2^n: exponential (does base matter?)

will often encounter them in future classes

Tools Options Help
Sans 12

Analysis of Sequential Search

Sequential Search

```
for i ← 0 to n − 1 do
    if list[i] == num
        return i
    end if
end for
return −1
```

- best case (e.g. *num* at 0): time $\Theta(1)$
- worst case (e.g. *num* at last or not found): time $\Theta(n)$

often just say $O(n)$-algorithm (linear complexity)

Layer: Layer 1

Tools Options Help
Sans 12

Analysis of Binary Search

Binary Search

```
left ← 0, right ← n − 1
while left ≤ right do
    mid ← floor((left + right)/2)
    if list[mid] > num
        left ← mid + 1
    else if list[mid] < num
        right ← mid − 1
    else
        return mid
    end if
end while
return −1
```

- best case (e.g. *num* at *mid*): time $\Theta(1)$
- worst case (e.g. *num* not found): because range $(right - left)$ halved in each WHILE, needs time $\Theta(\log n)$ iterations to decrease range to 0

often just say $O(\log n)$-algorithm (logarithmic complexity)

Layer: Layer 1

Tools Options Help
Sans 12

Analysis of Binary Search

Binary Search

```
left ← 0, right ← n − 1
while left ≤ right do
    mid ← floor((left + right)/2)
    if list[mid] > num
        left ← mid + 1
    else if list[mid] < num
        right ← mid − 1
    else
        return mid
    end if
end while
return −1
```

- best case (e.g. *num* at *mid*): time $\Theta(1)$
- worst case (e.g. *num* not found): because range $(right - left)$ halved in each WHILE, needs time $\Theta(\log n)$ iterations to decrease range to 0

often just say $O(\log n)$-algorithm (logarithmic complexity)

Layer: Layer 1

Sequential and Binary Search

- Input: any integer array *list* with size n, an integer *num*
- Output: if *num* not within *list*, -1; otherwise, $+1126$

DIRECT-SEQ-SEARCH (*list*, *n*, *num*)

```
for i ← 0 to n − 1 do
    if list[i] == num
        return +1126
    end if
end for
return −1
```

SORT-AND-BIN-SEARCH (*list*, *n*, *num*)

```
SEL-SORT(list, n)
return
BIN-SEARCH(list, n, num) ≥ 0? + 1126 : −1
```

- DIRECT-SEQ-SEARCH: $O(n)$ time
- SORT-AND-BIN-SEARCH: $O(n^2)$ time for SEL-SORT and $O(\log n)$ time for BIN-SEARCH

next: operations for "combining" asymptotic complexity

Tools Options Help
Sans 12

Some Properties of Big-O I

Theorem (封閉律)

if $f_1(n) = O(g_2(n))$, $f_2(n) = O(g_2(n))$ *then* $f_1(n) + f_2(n) = O(g_2(n))$

- When $n \geq n_1$, $f_1(n) \leq c_1 g_2(n)$
- When $n \geq n_2$, $f_2(n) \leq c_2 g_2(n)$
- So, when $n \geq \max(n_1, n_2)$, $f_1(n) + f_2(n) \leq (c_1 + c_2) g_2(n)$

Theorem (遞移律)

if $f_1(n) = O(g_1(n))$, $g_1(n) = O(g_2(n))$ *then* $f_1(n) = O(g_2(n))$

- When $n \geq n_1$, $f_1(n) \leq c_1 g_1(n)$
- When $n \geq n_2$, $g_1(n) \leq c_2 g_2(n)$
- So, when $n \geq \max(n_1, n_2)$, $f_1(n) \leq c_1 c_2 g_2(n)$

Layer: Layer 1

Some Properties of Big-O II

Theorem (併吞律)

if $f_1(n) = O(g_1(n))$, $f_2(n) = O(g_2(n))$ *and* $g_1(n) = O(g_2(n))$ *then* $f_1(n) + f_2(n) = O(g_2(n))$

Proof: use two theorems above.

Theorem

If $f(n) = a_m n^m + \cdots + a_1 n + a_0$, *then* $f(n) = O(n^m)$

Proof: use the theorem above.

similar proof for Ω and Θ

Some Properties of Big-O II

sel-sort is $O(n^2)$ bin-search is $O(\log n)$

f_1 g_1 f_2 g_2

Theorem (併吞律)

if $f_1(n) = O(g_1(n))$, $f_2(n) = O(g_2(n))$ *and* $g_1(n) = O(g_2(n))$ *then* $f_1(n) + f_2(n) = O(g_2(n))$

$\log n$ is $O(n^2)$

Proof: use two theorems above.

Theorem

If $f(n) = a_m n^m + \cdots + a_1 n + a_0$, *then* $f(n) = O(n^m)$

Proof: use the theorem above.

similar proof for Ω and Θ

$f_1 + f_2 = O(n^2)$

Tools Options Help
Sans 12

Some Properties of Big-O II

sel-sort is $O(n^2)$ bin-search is $O(\log n)$

f_2 g_1 f_1 g_2

Theorem (併吞律)

if $f_1(n) = O(g_1(n))$, $f_2(n) = O(g_2(n))$ *and* $g_1(n) = O(g_2(n))$ *then* $f_1(n) + f_2(n) = O(g_2(n))$

$\log n$ is $O(n^2)$

Proof: use two theorems above.

Theorem

If $f(n) = a_m n^m + \cdots + a_1 n + a_0$, *then* $f(n) = O(n^m)$

Proof: use the theorem above.

similar proof for Ω and Θ

$f_1 + f_2 = O(n^2)$

sel-sort-bin-search

Layer: Layer 1

Some More on Big-O

"$\leq$"

RECURSIVE-BIN-SEARCH is $O(\log n)$ time and $O(\log n)$ space

- by 遞移律, time also $O(n)$
- time also $O(n \log n)$
- time also $O(n^2)$
- also $O(2^n)$
- ...

prefer the tightest Big-O!

Tools Options Help
Sans 12

Stack: Last-In-First-Out (LIFO)

Stack

(constant-time) operations:

- `insertTop(data)`, often called `push(data)`
- `removeTop()`, often called `pop()`
- `getTop()`, often called `peek()`

—LIFO: 擠電梯， 洗盤子

very restricted data structure, but important for computers
—will discuss some cases later

Layer: Layer 1

Tools Options Help
Sans 12

A Simple Application: Parentheses Balancing

- in C, the following characters show up in pairs: (), [], {}, ""

```
good:  {xxx(xxxxxx)xxxxx"xxxx"x}
bad:   {xxx(xxxxxx}xxxxx"xxxx"x}
```

- the LISP programming language

```
(append (pow (* (+ 3 5) 2) 4) 3)
```

how can we check parentheses balancing?

Layer: Layer 1

Tools Options Help
Sans 12

Stack Solution to Parentheses Balancing

inner-most parentheses pair $\Longrightarrow$ top-most plate

'(': 堆盤子上去 ; ')': 拿盤子下來

Parentheses Balancing Algorithm

```
for each c in the input do
    if c is a left character
        push c to the stack
    else if c is a right character
        pop d from the stack and check if match
    end if
end for
```

many more sophisticated use in compiler design (will see some)

Layer: Layer 1

Tools Options Help
Sans 12

System Stack

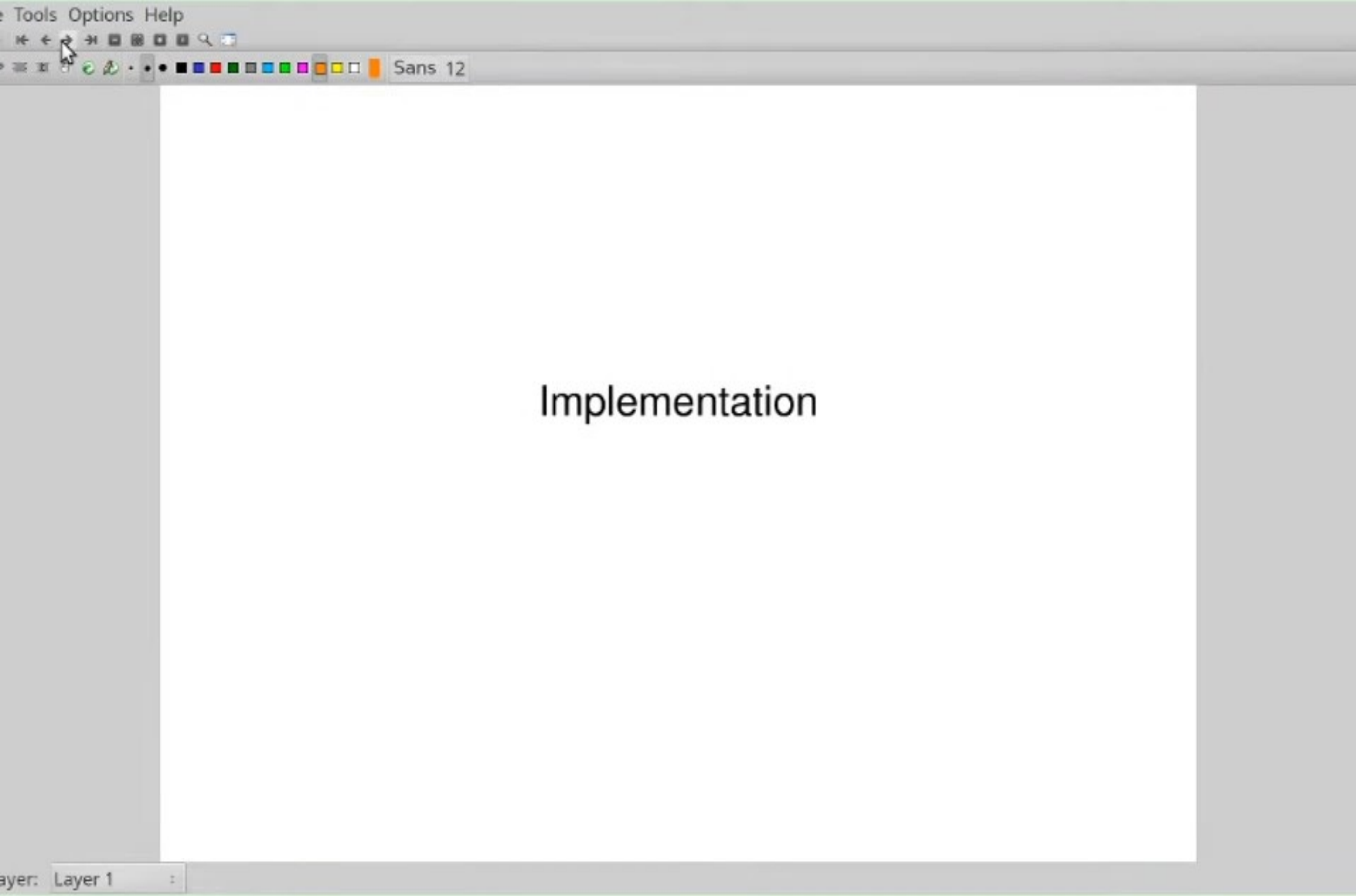
- recall: function call ⇔ 拿新的草稿紙來算
- old (original) scrap paper: temporarily not used, 可以壓在下面

System Stack: 一疊草稿紙, each paper (stack frame) contains

- return address: where to return to the previous scrap paper
- local variables (including parameters): to be used for calculating within this function
- previous frame pointer: to be used when escaping from this function

some related issues: security attack?

Layer: Layer 1


Tools Options Help
Sans 12
Implementation
Layer: Layer 1

Tools Options Help
Sans 12

Stacks Implemented on Array

usually: (growable) consecutive array and push/pop at `end-of-array`

Layer: Layer 1

Tools Options Help
Sans 12

Stacks Implemented on Linked List

usually: singly linked list and push/pop at `head`

Layer: Layer 1

Tools Options Help
Sans 12

Stack in STL

```
stack< int , vector<int> > s_on_array;
stack< int , list<int> > s_on_array;
```

list

implemented as container adapter

Layer: Layer 1